

1. Nazwa - Pełnomocnik

Pełnomocnik to strukturalny wzorzec projektowy pozwalający stworzyć obiekt zastępczy w miejsce innego obiektu. Pełnomocnik nadzoruje dostęp do pierwotnego obiektu, pozwalając na wykonanie jakiejś czynności przed lub po przekazaniu do niego żądania.

2. Problem

Po co właściwie kontrolować dostęp do obiektu? Załóżmy, że mamy duży obiekt, który zużywa dużo zasobów. Potrzebujesz go co jakiś czas, ale nie ciągle.

Moglibyśmy zastosować leniwą inicjalizację: tworzyć ten obiekt tylko gdy staje się faktycznie potrzebny. Wszystkie jego klienci musiałyby wykonać jakiś opóźniony kod inicjalizujący. Niestety doprowadziłoby to do powielania kodu.

W idealnym świecie, chcielibyśmy umieścić ten kod w klasie obiektu, ale nie zawsze jest to możliwe. Klasa może być bowiem częścią zamkniętej biblioteki innego dostawcy.

3. Rozwiązanie

Wzorzec Pełnomocnik zakłada stworzenie nowej klasy pośredniczącej, o takim samym interfejsie co pierwotny obiekt udostępniający usługę. Następnie aktualizujemy nasz program tak, aby przekazywał obiekt pełnomocnika wszystkim klientom pierwotnego obiektu. Otrzymawszy żądanie od klienta, pełnomocnik tworzy prawdziwy obiekt usługi i deleguje mu całą pracę.

Założenia:

Interfejs Usługi deklaruje interfejs z którym Pełnomocnik musi być zgodny, aby móc udawać obiekt usługi.

Usługa to klasa udostępniająca jakąś użyteczną logikę biznesową.

Klasa Pełnomocnik zawiera pole z odniesieniem do konkretnego obiektu udostępniającego usługę. Po wykonaniu swoich zadań (leniwa inicjalizacja, zapis w dzienniku, kontrola dostępu, przechowanie w pamięci podręcznej, itp.) Pełnomocnik przekazuje żądanie do tego obiektu.

Pośrednicy zazwyczaj zarządzają całym cyklem życia swych obiektów usługowych.

Klient powinien móc pracować zarówno z usługami, jak i pełnomocnikami za pośrednictwem takiego samego interfejsu. Dzięki temu można umieścić pełnomocnika w dowolnym kodzie, który ma współdziałać z obiektem usługowym.

Implementacja:

- Jeśli nie ma istniejącego interfejsu usługi, stwórz go, aby uczynić pełnomocnika i obiekt usługi wymiennymi. Ekstrakcja interfejsu z klasy usługi nie zawsze jest możliwa, ponieważ trzeba by było zmienić wszystkich klientów usługi tak, by komunikowali się przez ten interfejs. Plan B to stworzenie pełnomocnika w formie podklasy klasy usługi. Dzięki temu pełnomocnik odziedziczy interfejs usługi.
- Stwórz klasę pełnomocnika. Powinna posiadać pole służące do przechowywania odniesienia do usługi. Zazwyczaj pośrednicy zarządzają całym cyklem życia usługodawców, włącznie z tworzeniem ich obiektów. W pewnych przypadkach obiekt usługi jest przekazywany przez klienta pełnomocnikowi za pośrednictwem konstruktora.
- Zaimplementuj metody pełnomocnika zgodnie z ich przeznaczeniem. W większości przypadków, po wykonaniu jakiejś części pracy, pełnomocnik powinien oddelegować jej resztę obiektowi usługi.
- Rozważ utworzenie metody kreacyjnej która decyduje o tym, czy klient otrzyma obiekt pełnomocnika, czy faktyczny obiekt usługi. Może to być prosta, statyczna metoda w klasie pełnomocnika, albo w pełni rozwinięta metoda wytwórcza.
- Przemyśl implementację leniwej inicjalizacji obiektu usługi.

4. Konsekwencje

Zalety:

- Można sterować obiektem usługi bez wiedzy klientów.
- W przypadku aplikacji oszczędzamy pamięć i zasoby sprzętowe służące do realizacji ciężkich obliczeniowo operacji.
Można zarządzać cyklem życia obiektu usługi, gdy klientów to nie interesuje.
Pełnomocnik działa nawet wtedy, gdy obiekt udostępniający usługę nie jest jeszcze gotowy lub dostępny.
Zasada otwarte/zamknięte. Można wprowadzać nowych pełnomocników do aplikacji bez modyfikowania usług lub klientów.

Wady:

Kod może ulec skomplikowaniu, ponieważ trzeba wprowadzić wiele nowych klas. Odpowiedzi ze strony usługi mogą ulec opóźnieniu.